

通过统一代理配置 **Claude Code** 与 **Codex** 使用第三方模型

一个本机 HTTPS 入口，统一处理 Claude 槽位映射、Codex Responses API 桥接、证书信任与运行验证。

目标状态

Claude Code
Codex
统一接入 **Ark**
第三方模型

2026-05-14
标准方案介绍版

统一代理带来的四个优势。

方案核心不是工具各自配置，而是把入口、证书、模型策略、验证口径收敛到同一个本机 HTTPS 层。

统一入口

Claude Code 与 Codex 都指向
`https://127.0.0.1:38443`。

统一证书

本地 CA、Keychain 信任、host
loop 证书环境统一处理。

统一策略

Claude 保留槽位映射，Codex 直接
使用真实模型名。

统一验收

health、代理日志、profiles、tool
call 用同一套信号验证。

标准能力：一个代理完成协议适配、模型路由、证书信任和运行验证

目标架构：一个 **38443** 入口，两条协议分支。



本机 **HTTPS** 和证书信任是成败关键。

证书生成

本地 CA + server certificate ; SAN 包含 127.0.0.1 、 localhost 、 ::1 。

Keychain 信任

Desktop/Electron 不等同于 shell ，不能只依赖 `curl --cacert` 。

Cowork 兜底

host loop 环境不完整时，用 launcher 注入 `NODE_EXTRA_CA_CERTS` 。

判断原则 普通 `curl https://127.0.0.1:38443/health` 能成功，是系统信任链足够的关键证据。

统一代理需要实现四个能力。

1 健康检查

/health 返回 upstream 、
codexUpstream 、模型映射

2 Claude 透传

转发 Anthropic-compatible 请求，
透传认证头

3 模型映射

opus/sonnet/haiku ->
glm/kimi/doubao

4 Codex 桥接

Responses API -> Chat
Completions -> Responses API

API key 不写死在代理代码中，由客户端配置或环境变量提供

模型策略：Claude 映射，Codex 真实模型名。

工具 / 配置	客户端模型名	实际上游模型	策略	用途
Claude	claude-opus-4-6	glm-5.1	代理映射	复杂推理
Claude	claude-sonnet-4-6	kimi-k2.6	代理映射	默认主力
Claude	claude-haiku-4-5	doubao-seed-2.0-pro	代理映射	快速任务
Codex ark-doubao	doubao-seed-2.0-pro	doubao-seed-2.0-pro	真实模型	默认 / 快速
Codex ark-kimi	kimi-k2.6	kimi-k2.6	真实模型	复杂编码
Codex ark-glm	glm-5.1	glm-5.1	真实模型	高质量推理

Claude Code 配置重点：保留槽位名。

```
"ANTHROPIC_BASE_URL": "https://127.0.0.1:38443"  
"ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>"  
"ANTHROPIC_DEFAULT_OPUS_MODEL": "claude-opus-4-6"  
"ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4-6"  
"ANTHROPIC_DEFAULT_HAIKU_MODEL": "claude-haiku-4-5"  
"NODE_EXTRA_CA_CERTS": "<PROJECT_ROOT>/certs/ca.crt"
```

Desktop 3P Gateway

gatewayBaseUrl 指向 38443；inferenceModels 写 Claude 槽位。

CLI / host settings

删除 ANTHROPIC_MODEL 和 modelOverrides，避免双重映射。

Cowork

证书失败时用 launcher 注入 CA 环境。

Codex 配置重点：provider + profiles ◦

```
model_provider = "ark-coding"
model = "doubao-seed-2.0-pro"

[model_providers.ark-coding]
wire_api = "responses"
base_url = "https://127.0.0.1:38443/v1"
requires_openai_auth = true

[profiles.ark-kimi]
model = "kimi-k2.6"
```

默认

doubao-seed-2.0-pro , medium reasoning

编码

ark-kimi -> kimi-k2.6 , high reasoning

推理

ark-glm -> glm-5.1 , high reasoning

运行： **codex -p ark-doubao / ark-kimi / ark-glm**

Codex App 切模型：默认配置或启动覆盖。

CLI 支持 `-p profile`；App 启动入口更适合用 `~/.codex/config.toml` 顶层默认值或 `codex app -c` 覆盖。

CLI 临时切换

```
codex -p ark-kimi
codex -p ark-glm
```

App 默认切换

修改 `~/.codex/config.toml` 顶层 `model`，重启 App 或新开会话。

App 启动覆盖

```
codex app /path -c 'model="kimi-k2.6" ...
```

```
codex app /path/to/project \
  -c 'model_provider="ark-coding"' \
  -c 'model="kimi-k2.6"' \
  -c 'model_reasoning_effort="high"'
```

验证信号

代理日志出现
`codex responses model`
`kimi-k2.6 -> kimi-k2.6`

当前已打开会话不保证热切换；稳定验证请重启 **App** 或新开 **workspace**

验收矩阵：不要只看 UI，要看行为信号。

检查项	成功信号	状态
端口	38443 LISTEN ；旧 38444 不监听	OK
健康检查	/health 返回 ok 、 upstream 、 codexUpstream 、模型字段	OK
Claude Desktop	ConfigHealth recomputed { state: 'healthy', provider: 'gateway' }	OK
Claude 请求	POST /v1/messages -> 200 ；映射日志可见	OK
Codex profiles	ark-doubao / ark-kimi / ark-glm 均可回复	OK
Tool call	function_call 与 function_call_output 闭环正常	OK

运维流程：进程、端口、日志、配置、回滚。

1

进程

`launchctl print / kickstart`

2

端口

`lsof -nP -iTCP:38443`

3

健康

`curl -sk https://127.0.0.1:38443/health`

4

日志

`proxy.log / proxy.err.log / Claude main.log`

5

回滚

恢复 `server.js` 与配置备份

故障优先级

1. health 是否通
2. 证书是否被系统信任
3. 请求是否进入正确分支
4. Authorization 是否存在
5. tool call 转换是否完整

安全边界：材料可分享，密钥不进材料。

密钥

真实 API key 只进入本机配置或环境变量，文档统一使用占位符。

证书

certs/*.key 不公开、不入库；迁移机器优先重新生成。

日志

公开前扫描 authorization 、 x-api-key 、 cookie 、个人路径。

公司批量部署建议

CA test 信任通过 **MDM** 配置描述文件下发；安装脚本只负责代理、**LaunchAgent**、工具配置和 **smoke**

标准化交付：一份 **PPT**、一份技术手册、一份 **AI runbook**。

介绍 **PPT**

说明统一代理优势、架构如何工作、Claude/Codex 如何分别配置。

技术手册 **Word**

面向人类维护者，覆盖部署、配置、验证、运维、回滚、安全。

AI Runbook

面向其他 AI 工具，使用占位符、步骤、检查点和输出格式。

下一步：把远端已验证的合并版 **server.js** 同步回材料仓库，并补自动 **smoke test**